

Ring VRF

*A cryptographic primitive for anonymous,
accountable identity*

Torsten Stüber | Parity

PBA Lisbon, April 2026

Ground rules for this lecture

What this is

- An intuition-first tour of a real cryptographic primitive
- Four building blocks, assembled into one thing

What this is not

- A proof of anything
- A construction you could implement from these slides

Ask questions. Interrupt.

Meet Alice



Meet Alice

1. Enrolls

2. Votes in Referendum 42

3. Comments on Subsquare post 17



Meet Alice

1. Enrolls

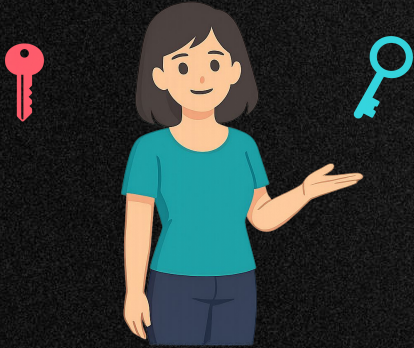
2. Votes in Referendum 42

3. Comments on Subsquare post 17

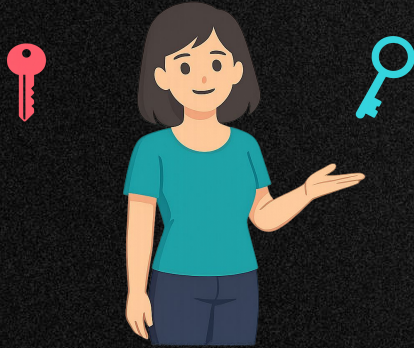


Nobody ever learns it's Alice.
And she won't be able to cheat.

Alice enrolls



Alice enrolls



Decentralized Identity Modules (DIMs)

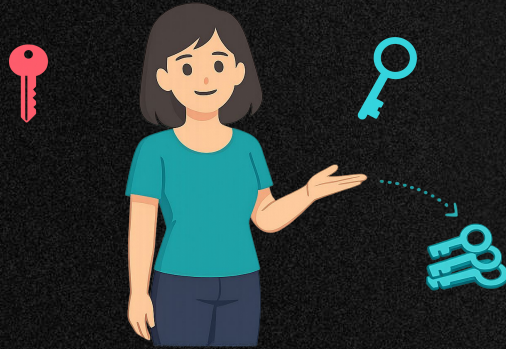


In-person,
time-synchronised
meetup



Proof of ink
Painful to Sybil

Alice enrolls



Decentralized Identity Modules (DIMs)



In-person,
time-synchronised
meetup



Proof of ink
Painful to Sybil

Alice enrolls



Decentralized Identity Modules (DIMs)

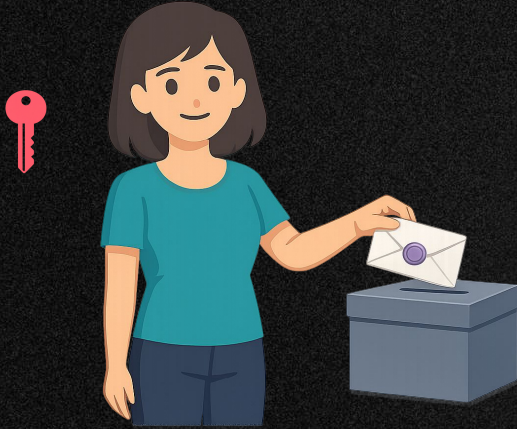


In-person,
time-synchronised
meetup

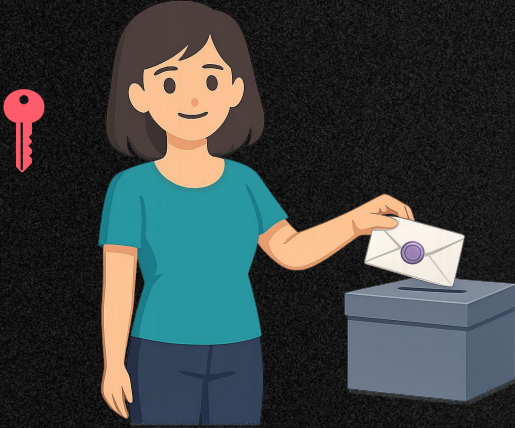


Proof of ink
Painful to Sybil

Alice votes

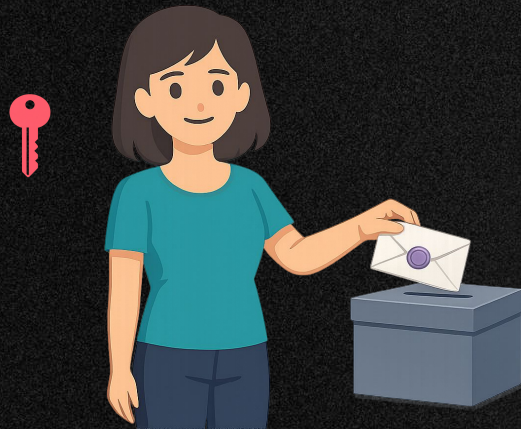


Alice votes

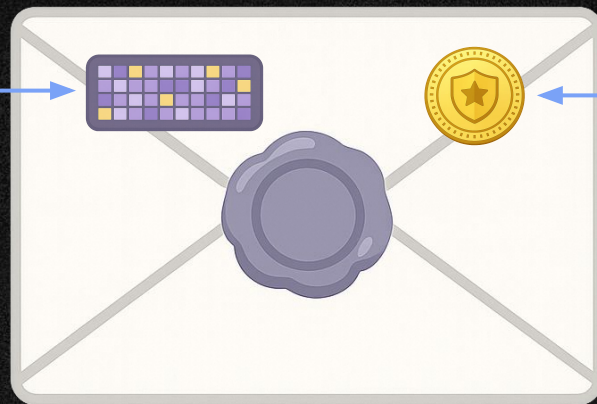


```
context = "referendum-42"
```


Alice votes



context = "referendum-42"

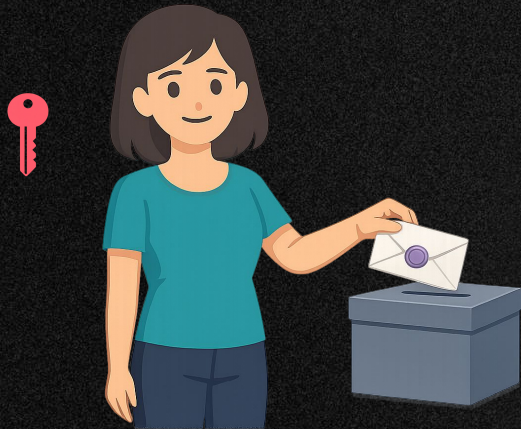


Pseudonym

alias_42

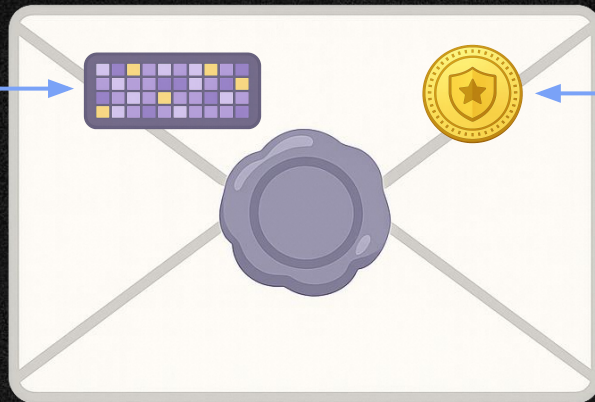
SNARK proof
Few hundred
bytes

Alice votes



the same Alice + same
context always produces
the same `alias_42`
so she can't vote twice.

`context = "referendum-42"`

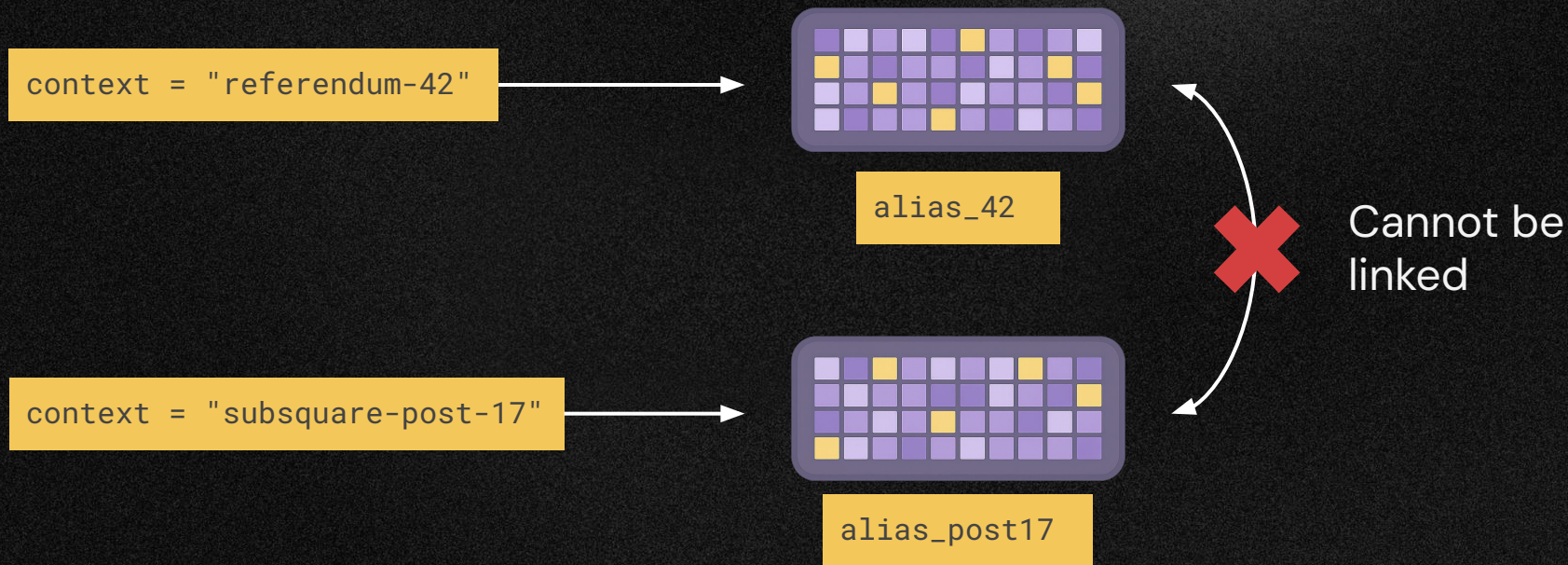


Pseudonym

`alias_42`

SNARK proof
Few hundred
bytes

Alice comments



What we need

1. **Membership**: Alice's key is in the registered set
→ property: anonymity within a set
2. **Determinism per context**: same person + same context → same alias
→ property: Sybil-resistance within a context
3. **Unlinkable across contexts**: different contexts → independent aliases
→ property: cross-context privacy
4. **Succinct & publicly verifiable**
→ property: the chain can check it cheaply

What we need

1. **Membership**: Alice's key is in the registered set
→ property: anonymity within a set
2. **Determinism per context**: same person + same context → same alias
→ property: Sybil-resistance within a context
3. **Unlinkable across contexts**: different contexts → independent aliases
→ property: cross-context privacy
4. **Succinct & publicly verifiable**
→ property: the chain can check it cheaply

No single primitive in a textbook gives you all four. Ring VRF is the primitive that does.

Roadmap: four tricks

Trick 1: The magic lottery ticket

Verifiable Random Functions

Gives us determinism + verifiability

Roadmap: four tricks

Trick 1: The magic lottery ticket

Verifiable Random Functions

Gives us determinism + verifiability

Trick 2: Sealed Envelopes

Cryptographic commitments

The glue

Roadmap: four tricks

Trick 1: The magic lottery ticket

Verifiable Random Functions

Gives us determinism + verifiability

Trick 2: Sealed Envelopes

Cryptographic commitments

The glue

Trick 2: Hiding in crowd

Ring membership via polynomial
commitments + SNARKs

Gives us anonymity

Roadmap: four tricks

Trick 1: The magic lottery ticket

Verifiable Random Functions

Gives us determinism + verifiability

Trick 2: Sealed Envelopes

Cryptographic commitments

The glue

Trick 2: Hiding in crowd

Ring membership via polynomial commitments + SNARKs

Gives us anonymity

Trick 4: Assembly

Staple them all into one proof

Gives us Ring VRF

Section 2

Trick 1: The magic lottery
ticket

Something familiar: signatures



Signer has **sk** 
Takes message
Produces signature



Verifier has **PK** , message, signature
Outputs valid/invalid

- Signatures can be randomized: many valid signatures per message
- Signatures don't output a value, the message is already known
- We want something more

VRF: the deterministic lottery ticket

A permissionless blockchain: who produces the next block?



VRF: the deterministic lottery ticket

A permissionless blockchain: who produces the next block?



At slot N , each validator computes $\text{ticket} = \text{VRF}(\text{sk}, N)$

VRF: the deterministic lottery ticket

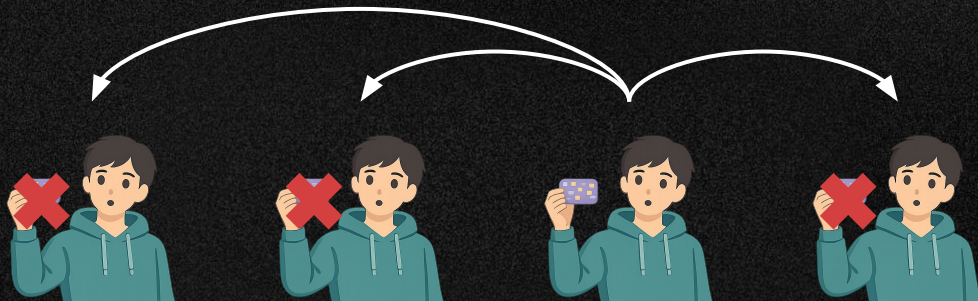
A permissionless blockchain: who produces the next block?



At slot N , each validator computes $\text{ticket} = \text{VRF}(\text{sk}, N)$

VRF: the deterministic lottery ticket

A permissionless blockchain: who produces the next block?



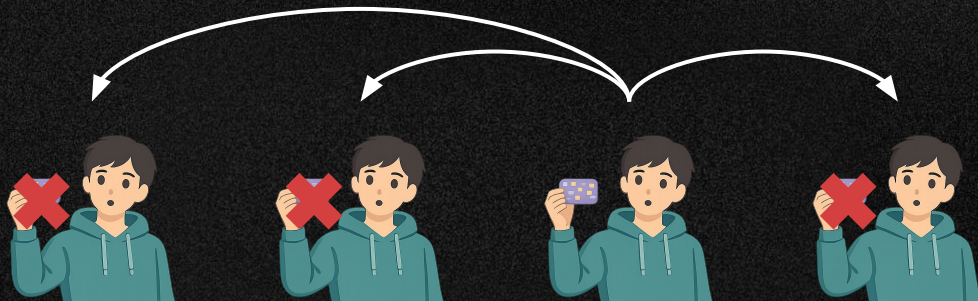
At slot N , each validator computes $\text{ticket} = \text{VRF}(\text{sk}, N)$

The leader publishes (ticket, proof) to the network

Anyone with the leader's PK verifies

VRF: the deterministic lottery ticket

A permissionless blockchain: who produces the next block?



At slot N , each validator computes $\text{ticket} = \text{VRF}(\text{sk}, N)$

The leader publishes $(\text{ticket}, \text{proof})$ to the network

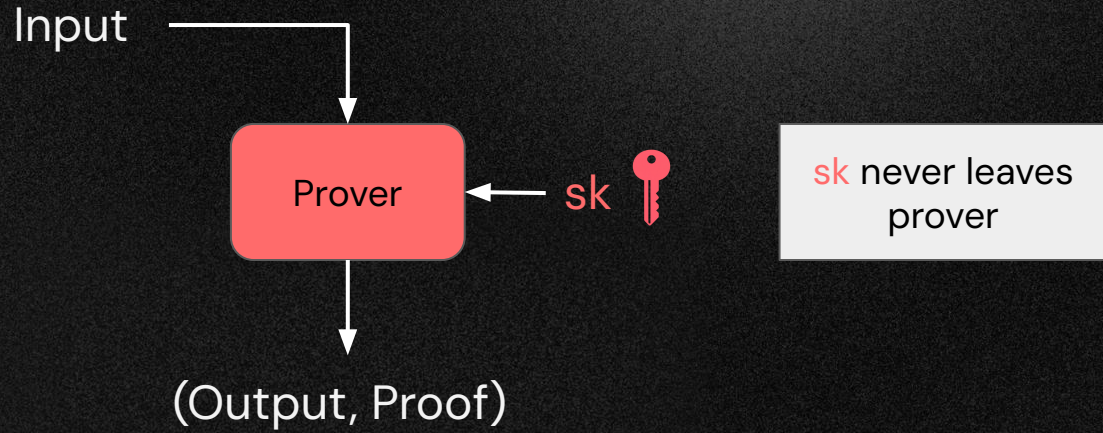
Anyone with the leader's PK verifies

Used in Algorand, Cardano Praos, Ethereum beacon chain, Polkadot BABE/SASSAFRAS

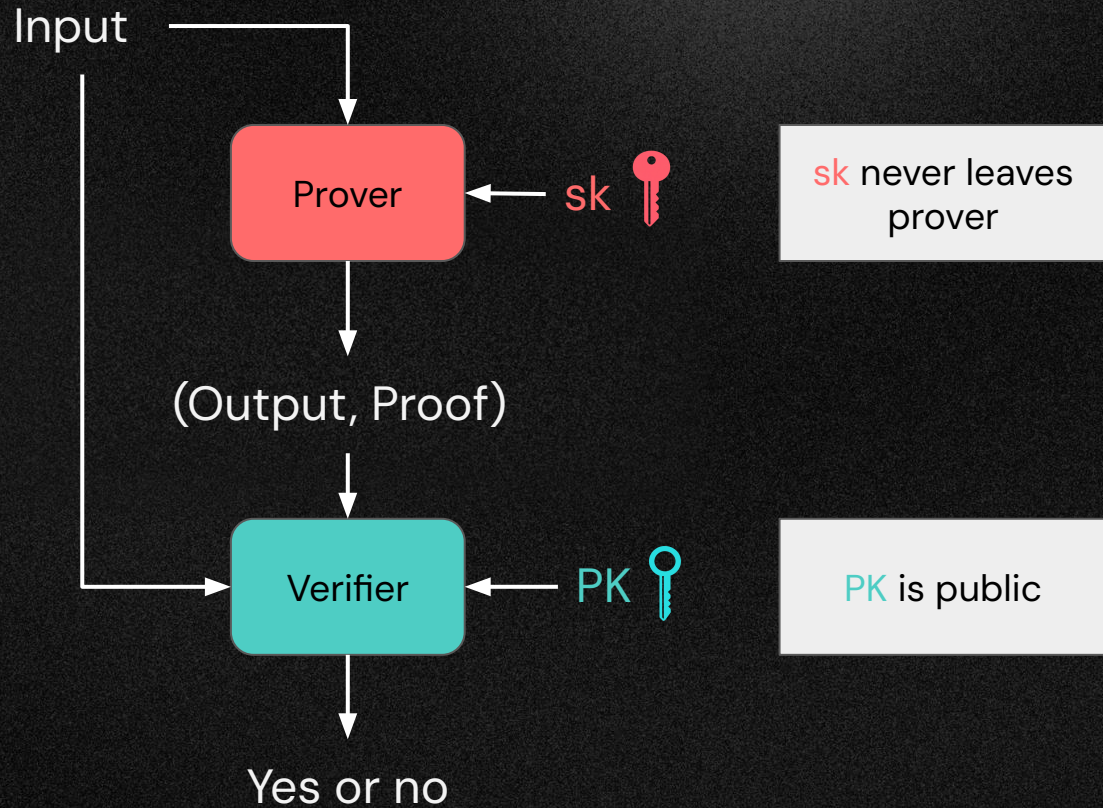
The Verifiable Random Function

Property	What it means
Deterministic	Same sk , same input → same output, every time
Pseudorandom	Without sk (and without seeing a proof), the output is indistinguishable from random
Verifiable	The prover produces a short proof alongside the output. With PK , the input, the output, and the proof, anyone can check correctness.
Unique	For each (PK , input), at most one valid output exists

VRF interface



VRF interface



What you need to believe about elliptic curves

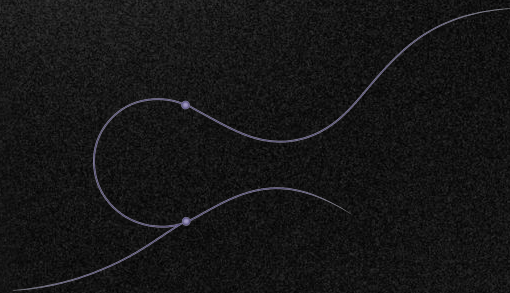
It's an algebra

- A finite set whose elements we'll call **points**
We don't care what they look like
- A **+** operation: two points $\rightarrow$ a third point
Associative, commutative, identity, inverses
(Abelian group)
- A distinguished **base point G**
A public reference point every protocol using the curve agrees on

What you need to believe about elliptic curves

It's an algebra

- A finite set whose elements we'll call **points**
We don't care what they look like
- A **+** operation: two points $\rightarrow$ a third point
Associative, commutative, identity, inverses
(Abelian group)
- A distinguished **base point G**
A public reference point every protocol using the curve agrees on



Bandersnatch

What you need to believe about elliptic curves

Scalar multiplication is repeated addition

- **scalar**: ordinary integer
(taken modulo large prime p , for Bandersnatch, p is about 253 bits)
- Scalar arithmetic is field arithmetic mod p : add, subtract, multiply, and divide

What you need to believe about elliptic curves

Scalar multiplication is repeated addition

- **scalar**: ordinary integer
(taken modulo large prime p , for Bandersnatch, p is about 253 bits)
- Scalar arithmetic is field arithmetic mod p : add, subtract, multiply, and divide
- $k \cdot P$ means $P + P + \dots + P$, k times

What you need to believe about elliptic curves

Scalar multiplication is repeated addition

- **scalar**: ordinary integer
(taken modulo large prime p , for Bandersnatch, p is about 253 bits)
- Scalar arithmetic is field arithmetic mod p : add, subtract, multiply, and divide
- $k \cdot P$ means $P + P + \dots + P$, k times
- Convention:
 - uppercase letter = point on curve
 - lowercase = scalar

What you need to believe about elliptic curves

Useful asymmetry

Direction	Cost
Given scalar k and point P , compute $k \cdot P$	Fast: Microseconds even for 256-bit k
Given points P, Q , compute $P + Q$	Fast: A handful of field operations
Given $P = k \cdot G$, recover k	Intractable: No known efficient algorithm

What you need to believe about elliptic curves

Useful asymmetry

Direction	Cost
Given scalar k and point P , compute $k \cdot P$	Fast: Microseconds even for 256-bit k
Given points P, Q , compute $P + Q$	Fast: A handful of field operations
Given $P = k \cdot G$, recover k	Intractable: No known efficient algorithm



Discrete logarithm problem

k always exists and is unique

What you need to believe about elliptic curves

Useful asymmetry

Direction	Cost
Given scalar k and point P , compute $k \cdot P$	Fast: Microseconds even for 256-bit k
Given points P, Q , compute $P + Q$	Fast: A handful of field operations
Given $P = k \cdot G$, recover k	Intractable: No known efficient algorithm



Discrete logarithm problem

k always exists and is unique

**All elliptic-curve cryptography rests
on discrete logarithm hardness**

How to build a VRF: an elliptic-curve construction

- G is the public base point on an elliptic curve
- Alice's keypair: sk is a secret scalar, $PK = sk \cdot G$

How to build a VRF: an elliptic-curve construction

- G is the public base point on an elliptic curve
- Alice's keypair: sk is a secret scalar, $PK = sk \cdot G$
- To evaluate on input msg :
 - $H(msg)$: hash the input onto the curve $\rightarrow$ a point P
 - $Out = sk \cdot P$

How to build a VRF: an elliptic-curve construction

- G is the public base point on an elliptic curve
- Alice's keypair: sk is a secret scalar, $PK = sk \cdot G$
- To evaluate on input msg :
 - $H(msg)$: hash the input onto the curve $\rightarrow$ a point P
 - $Out = sk \cdot P$
- Alice proves the statement:
 - "The same sk that makes $PK = sk \cdot G$ also makes $Out = sk \cdot H(msg)$ "
 - Called *DLEQ* (*discrete log equality proof*)

How to build a VRF: an elliptic-curve construction

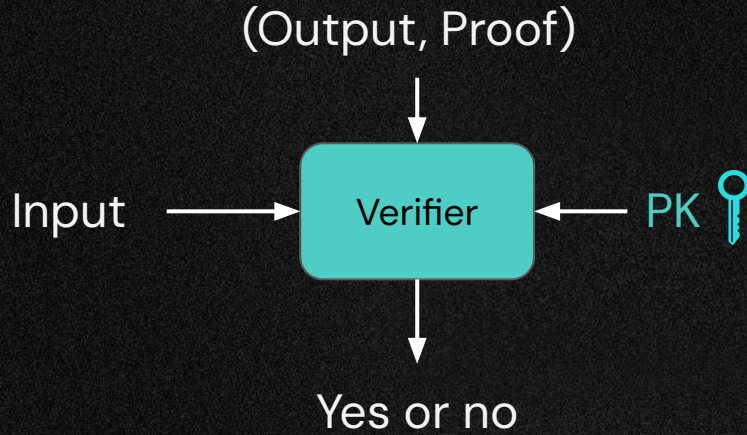
- G is the public base point on an elliptic curve
- Alice's keypair: sk is a secret scalar, $PK = sk \cdot G$
- To evaluate on input msg :
 - $H(msg)$: hash the input onto the curve $\rightarrow$ a point P
 - $Out = sk \cdot P$
- Alice proves the statement:
 - "The same sk that makes $PK = sk \cdot G$ also makes $Out = sk \cdot H(msg)$ "
 - Called *DLEQ* (*discrete log equality proof*)

That's it. That's a VRF.

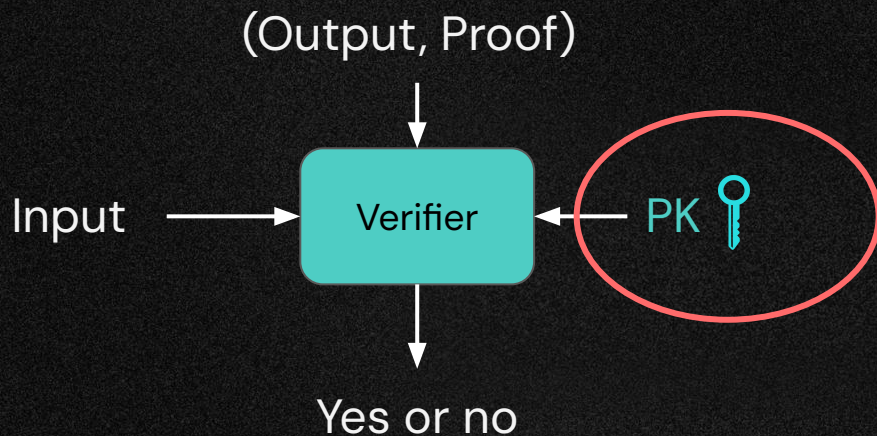
Why this satisfies our requirements

Requirement	Why it holds
Deterministic	$sk \cdot H(msg)$ has no randomness
Pseudorandom	Looks like uniformly random curve point (DDH)
Verifiable	DLEQ proof + PK makes the check possible for anyone
Unique	Group operation is a function (one input, one output)

But there's a problem



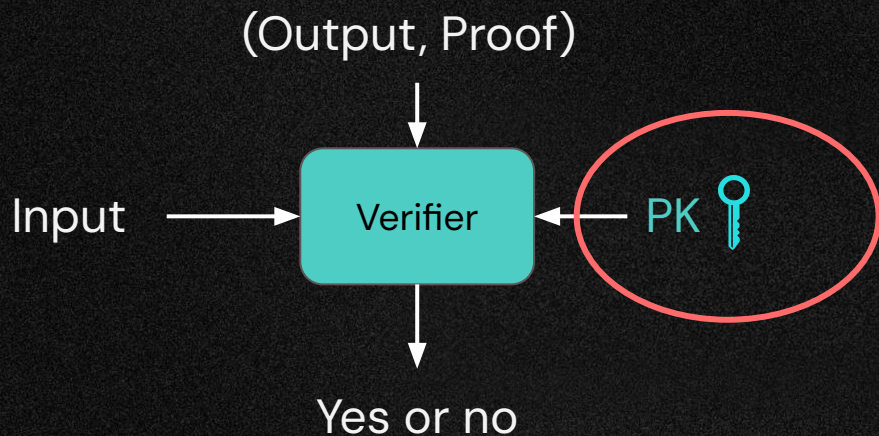
But there's a problem



To verify the VRF, the verifier needs the prover's public key

But the prover doesn't want to be identified

But there's a problem



To verify the VRF, the verifier needs the prover's public key

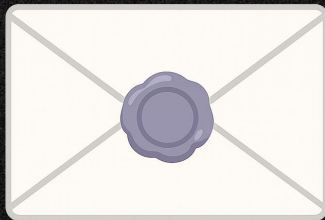
But the prover doesn't want to be identified

We need to hide which public key was used, while still allowing verification

The path forward

We will

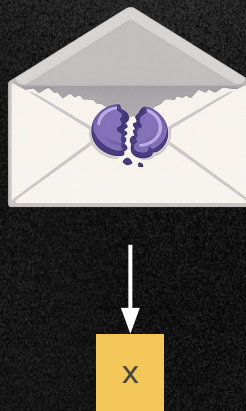
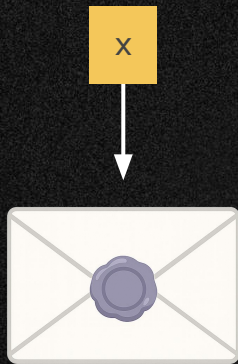
- hide Alice's **sk** inside a sealed envelope (Trick 2), and
- prove that envelope's contents lies in the ring of registered keys (Trick 3)
- without revealing which key is inside



Section 3

Trick 2: Sealed envelopes

What is a commitment?



- **Hiding:** The sealed envelope reveals nothing about x
- **Binding:** Once sealed, the prover can't change x

A first commitment, the easy way: hash-based

- $C = \text{Hash}(x)$
 - Hash: any cryptographic hash function (SHA-256, Blake2, ...)
 - x : value being committed

A first commitment, the easy way: hash-based

- $C = \text{Hash}(x)$
 - Hash: any cryptographic hash function (SHA-256, Blake2, ...)
 - x : value being committed
- Problem
 - Same $x \rightarrow$ same C
 - If space of possible x values is small, verifier can guess by re-hashing

A first commitment, the easy way: hash-based

- $C = \text{Hash}(x \parallel r)$
 - x : value being committed
 - r : fresh random scalar (the blinding factor)

A first commitment, the easy way: hash-based

- $C = \text{Hash}(x \parallel r)$
 - x : value being committed
 - r : fresh random scalar (the blinding factor)
- Two properties:
 - **Hiding**: Hash behaves like a random function: without knowing r , C leaks nothing about x
 - **Binding**: To open as a different (x', r') , Alice would need a hash collision
Infeasible for strong hashes

A first commitment, the easy way: hash-based

- $C = \text{Hash}(x \parallel r)$
 - x : value being committed
 - r : fresh random scalar (the blinding factor)
- Two properties:
 - **Hiding**: Hash behaves like a random function: without knowing r , C leaks nothing about x
 - **Binding**: To open as a different (x', r') , Alice would need a hash collision
Infeasible for strong hashes
- Opening: To reveal, Alice sends (x, r)
Verifier recomputes $\text{Hash}(x \parallel r)$ and checks it matches C

Pedersen commitments

Fixed: base point G , another point H

Nobody knows t with $H = t \cdot G$

Pedersen commitments

Fixed: base point G , another point H

Nobody knows t with $H = t \cdot G$

Commit to x : pick random blinding factor r , output

$$C = x \cdot G + r \cdot H$$

Pedersen commitments

Fixed: base point G , another point H

Nobody knows t with $H = t \cdot G$

Commit to x : pick random blinding factor r , output

$$C = x \cdot G + r \cdot H$$

- Hiding: r is uniform $\Rightarrow C$ is uniformly distributed on the curve
- Binding: To reveal a different x' ,
Alice would need to know $\log_G(H)$.

Hard!

What we'll use commitments for

Commit-then-reveal

Alice commits to x

Later opens x

vs

Commit-then-prove

Alice commits to x

Never reveals it

Commitment is handle for proofs

What we'll use commitments for

Commit-then-reveal

Alice commits to x

Later opens x

vs

Commit-then-prove

Alice commits to x

Never reveals it

Commitment is handle for proofs



C: Public object. Goes on-chain.
Never opened.

$$C = \overset{\text{key}}{\text{sk}} \cdot G + r \cdot H$$

What we'll use commitments for

C: Public object. Goes on-chain.
Never opened.

Commit-then-reveal

Alice commits to x
Later opens x

vs

Commit-then-prove

Alice commits to x
Never reveals it
Commitment is handle for proofs



$$C = \overset{\text{key}}{\text{sk}} \cdot G + r \cdot H$$

VRF evaluation Proof

"The sk in C
produced the VRF
output"

Ring membership Proof

"The sk in C
corresponds to a key
in the ring"

What we'll use commitments for

C: Public object. Goes on-chain.
Never opened.

Commit-then-reveal

Alice commits to x
Later opens x

vs

Commit-then-prove

Alice commits to x
Never reveals it
Commitment is handle for proofs



$$C = \overset{\text{key}}{\text{sk}} \cdot G + r \cdot H$$

VRF evaluation Proof

"The sk in C
produced the VRF
output"

Ring membership Proof

"The sk in C
corresponds to a key
in the ring"

Hiding: Nobody can extract sk from C

Binding: Alice can't swap sk , it is the same in
both proofs, even though never revealed

Section 4

Trick 3: Hiding in the crowd

Ring signatures: the original idea



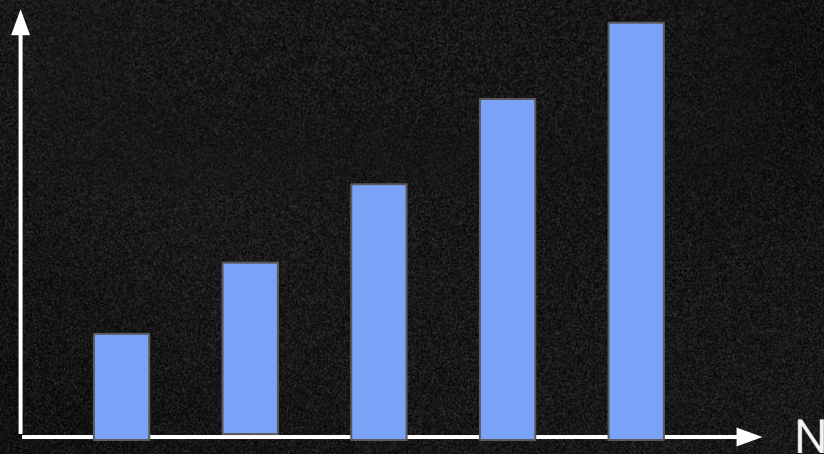
Ring signatures
(Rivest, Shamir, Tauman 2001).

Canonical use-case: a
whistleblower inside a cabinet.

Naive ring signature: OR of Schnorr

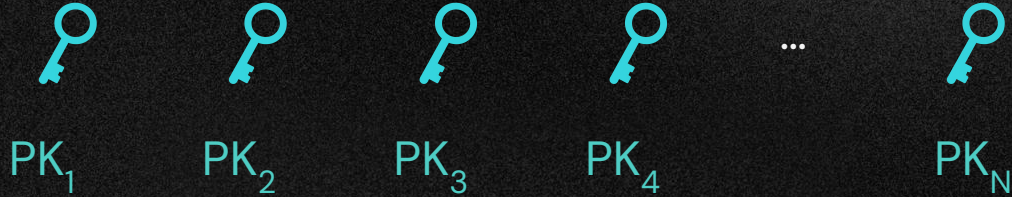
Prove I know sk_1 OR sk_2 OR ... OR sk_N

Proof size

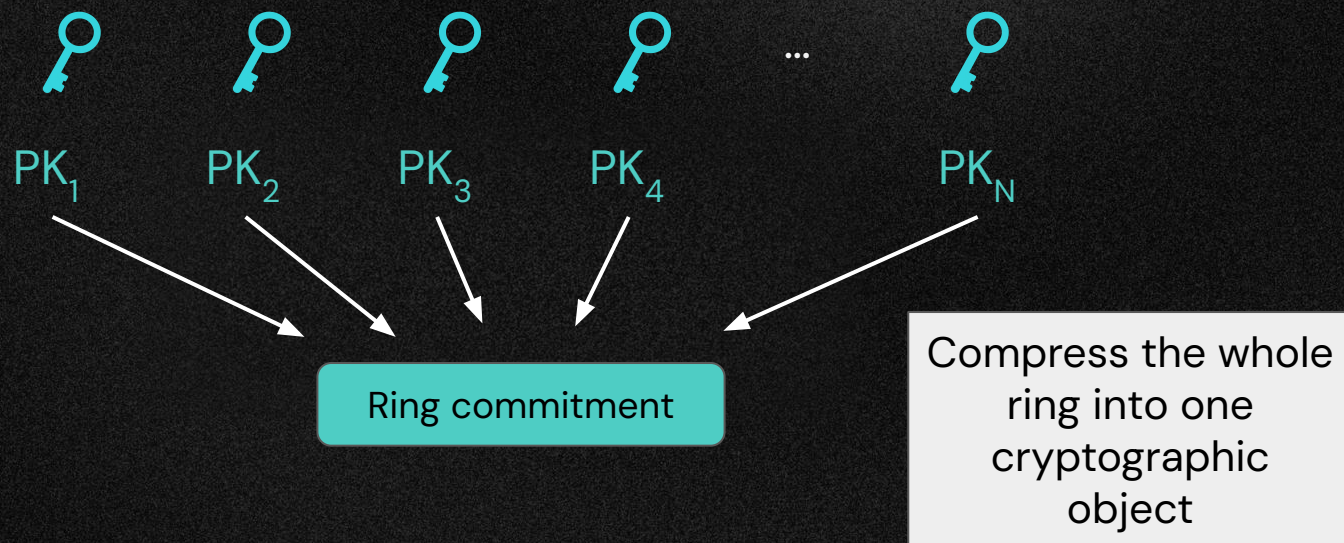


Not scalable

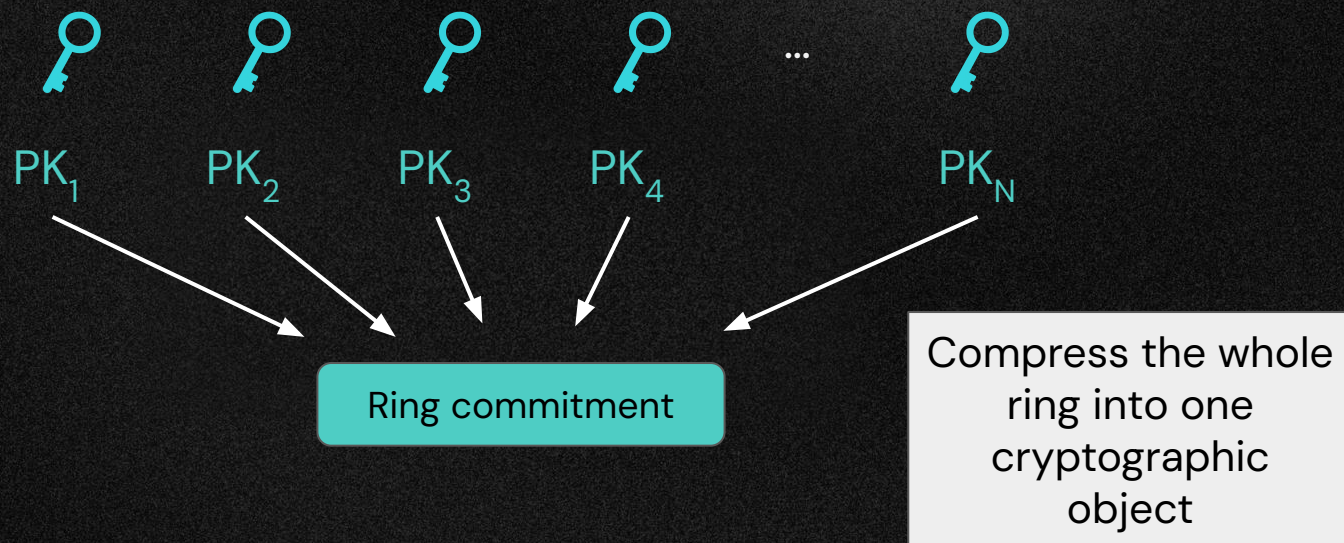
The key insight



The key insight



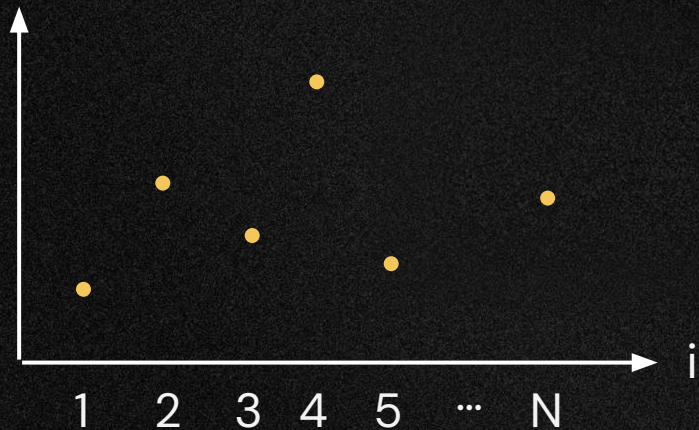
The key insight



Proofs reference the commitment, not the list

How? Polynomial commitments

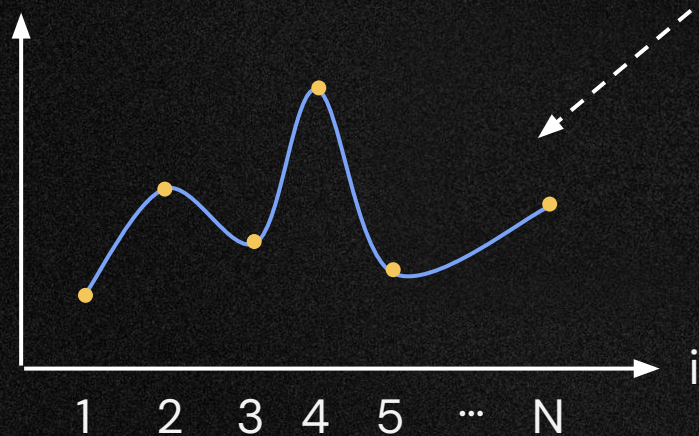
Scalar encoding of PK_i



How? Polynomial commitments

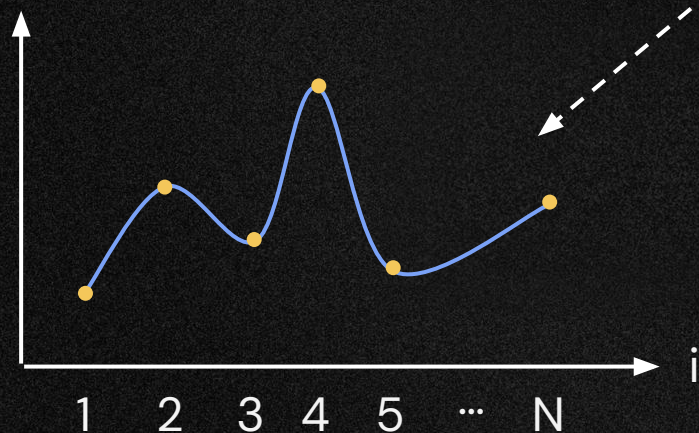
Scalar encoding of PK_i

Polynomial f with $f(i) = \text{encode}(PK_i)$



How? Polynomial commitments

Scalar encoding of PK_i



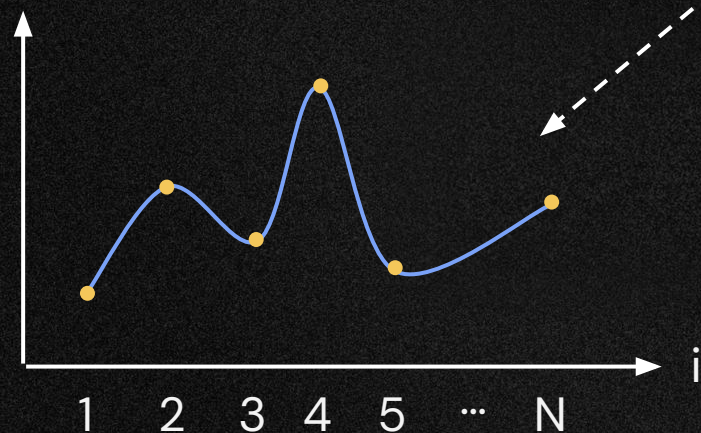
Polynomial f with $f(i) = \text{encode}(PK_i)$



KGZ Commitment
48 Byte curve point
that commits f

How? Polynomial commitments

Scalar encoding of PK_i



KGZ Commitment
48 Byte curve point
that commits f

To prove $f(i) = \text{encode}(PK_i)$, send tiny proof that opens commitment at i

Membership alone isn't enough

I know the secret key for some PK in the ring.

What we want to prove



$PK = PK_{17}$

What a naive KZG opening would leak

Zero-knowledge proofs: a Sudoku example

1	4							7
5	9			1			2	
3		6			8		4	1
4			9		5			
	6	5	4	8		1	9	3
2		9	7					4
								9
9			2					
6	5	4		9	3		1	2

Alice claims she solved the puzzle

Zero-knowledge proofs: a Sudoku example

1	4							7
5	9			1			2	
3		6			8		4	1
4			9		5			
	6	5	4	8		1	9	3
2		9	7					4
								9
9			2					
6	5	4		9	3		1	2

Statement x (public):
"This puzzle has a valid solution."

1	4	8	3	2	9	5	6	7
5	9	7	6	1	4	3	2	8
3	2	6	5	7	8	9	4	1
4	8	1	9	3	5	2	7	6
7	6	5	4	8	2	1	9	3
2	3	9	7	6	1	8	5	4
8	7	2	1	5	6	4	3	9
9	1	3	2	4	7	6	8	5
6	5	4	8	9	3	7	1	2

Witness w : A complete filling

Zero-knowledge proofs: a Sudoku example

1	4							7
5	9			1			2	
3		6			8		4	1
4			9		5			
	6	5	4	8		1	9	3
2		9	7					4
								9
9			2					
6	5	4		9	3		1	2

1	4	8	3	2	9	5	6	7
5	9	7	6	1	4	3	2	8
3	2	6	5	7	8	9	4	1
4	8	1	9	3	5	2	7	6
7	6	5	4	8	2	1	9	3
2	3	9	7	6	1	8	5	4
8	7	2	1	5	6	4	3	9
9	1	3	2	4	7	6	8	5
6	5	4	8	9	3	7	1	2

Two things Alice wants

- **Convince:** the verifier gets convinced that there is a witness **w**
- **Zero-knowledge:** the verifier learns nothing about **w**

Zero-knowledge proofs: a Sudoku example

1	4	8	3	2	9	5	6	7
5	9	7	6	1	4	3	2	8
3	2	6	5	7	8	9	4	1
4	8	1	9	3	5	2	7	6
7	6	5	4	8	2	1	9	3
2	3	9	7	6	1	8	5	4
8	7	2	1	5	6	4	3	9
9	1	3	2	4	7	6	8	5
6	5	4	8	9	3	7	1	2



1	2	3	4	5	6	7	8	9
								

Zero-knowledge proofs: a Sudoku example

1	4	8	3	2	9	5	6	7
5	9	7	6	1	4	3	2	8
3	2	6	5	7	8	9	4	1
4	8	1	9	3	5	2	7	6
7	6	5	4	8	2	1	9	3
2	3	9	7	6	1	8	5	4
8	7	2	1	5	6	4	3	9
9	1	3	2	4	7	6	8	5
6	5	4	8	9	3	7	1	2

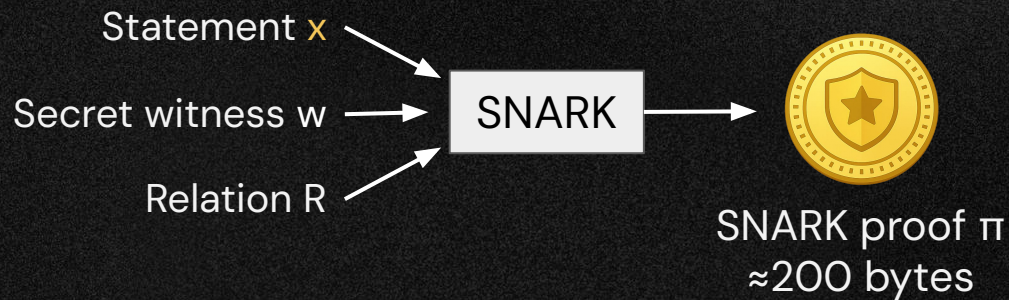
Two things Alice wants

- **Convince:** the verifier gets convinced that there is a witness w
- **Zero-knowledge:** the verifier learns nothing about w

- Given some $w \rightarrow$ verify in polynomial time
check every row, column, and 3×3 block
- Finding some w from the puzzle alone
 $\rightarrow$ NP-hard
- Sudoku is a canonical NP problem

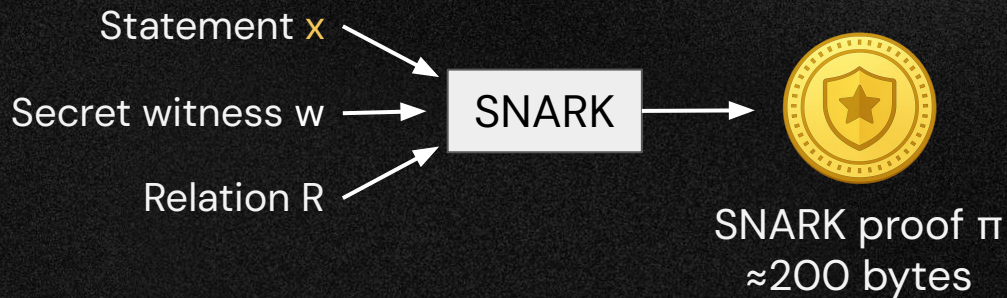
SNARKs as a black box

Succinct Non-Interactive
Argument of Knowledge



SNARKs as a black box

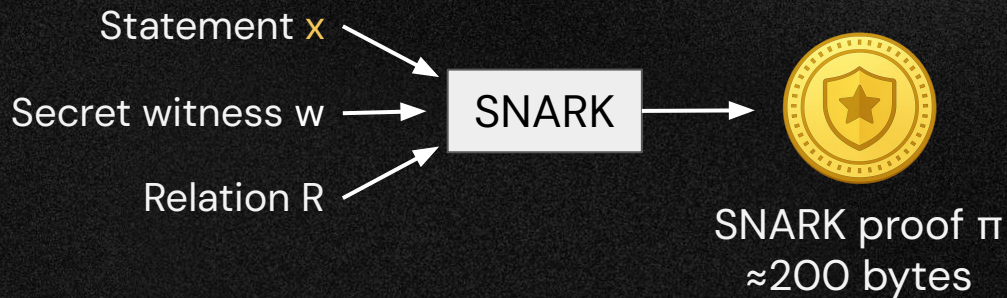
Succinct Non-Interactive
Argument of Knowledge



- **Succinct:** π is ~ 200 bytes regardless of how complex R is
- **Fast to verify:** a few milliseconds, constant time
- **Zero-knowledge:** π reveals nothing about w beyond "a valid w exists."

SNARKs as a black box

Succinct Non-Interactive
Argument of Knowledge



- **Succinct:** π is ~ 200 bytes regardless of how complex R is
- **Fast to verify:** a few milliseconds, constant time
- **Zero-knowledge:** π reveals nothing about w beyond "a valid w exists."

The construction is genuinely a semester-long topic

SNARK API

$\text{setup}(R) \rightarrow (\text{prove_key}, \text{verify_key})$

one-time, per relation

$\text{prove}(\text{prove_key}, x, w) \rightarrow \pi$

secret-witness holder
 π reveals nothing about w

$\text{verify}(\text{verify_key}, x, \pi) \rightarrow \checkmark / \times$

anyone; constant time

Our ring-membership SNARK

- Statement (public): ring root C_{ring} , commitment C_{sk}

Our ring-membership SNARK

- Statement (public): ring root C_{ring} , commitment C_{sk}
- Witness (secret): sk , position i , opening proof for the ring at position i

Our ring-membership SNARK

- Statement (public): ring root C_{ring} , commitment C_{sk}
- Witness (secret): sk , position i , opening proof for the ring at position i
- Relation to prove:
 - $C_{\text{sk}} = sk \cdot G + r \cdot H$
i.e.: commitment correctly hides sk
 - opening of C_{ring} at position i equals $sk \cdot G$
i.e.: the key behind the commitment is in the ring

Our ring-membership SNARK

- Statement (public): ring root C_{ring} , commitment C_{sk}
- Witness (secret): sk , position i , opening proof for the ring at position i
- Relation to prove:
 - $C_{\text{sk}} = sk \cdot G + r \cdot H$
i.e.: commitment correctly hides sk
 - opening of C_{ring} at position i equals $sk \cdot G$
i.e.: the key behind the commitment is in the ring
- Output: SNARK proof, constant size

Section 5

Assembly

Ring VRF = VRF + ring membership + commitment glue

Pedersen commitment

$$C_{sk} = sk \cdot G + r \cdot H$$

Ring VRF = VRF + ring membership + commitment glue

Pedersen commitment

$$C_{sk} = sk \cdot G + r \cdot H$$

VRF evaluation

proves

$$\text{Out} = sk \cdot H(\text{msg}),$$

given C_{sk}

Ring membership

proves

$$sk \cdot G \text{ appears in } C_{\text{ring}},$$

given C_{sk}

Ring VRF = VRF + ring membership + commitment glue

Pedersen commitment

$$C_{sk} = sk \cdot G + r \cdot H$$

VRF evaluation

proves
 $Out = sk \cdot H(msg)$,
given C_{sk}

Ring membership

proves
 $sk \cdot G$ appears in C_{ring} ,
given C_{sk}

Ring VRF signature

The formal relation

Public: msg, C_sk, C_ring, Out

Witness: sk, r, position i, ring opening at i

Relation:

$$C_sk = sk \cdot G + r \cdot H$$

$$Out = sk \cdot H(msg)$$

$$sk \cdot G = \text{OpenRing}(C_ring, \text{position } i)$$

The formal relation

Public: msg, C_sk, C_ring, Out

Witness: sk, r, position i, ring opening at i

Relation:

$$C_{sk} = sk \cdot G + r \cdot H$$

$$Out = sk \cdot H(msg)$$

$$sk \cdot G = \text{OpenRing}(C_{ring}, \text{position } i)$$

Full signature:

C_sk, proof_ring, proof_VRF

≈700 bytes

What we actually built

Requirement	Delivered by...
Anonymity within a registered set	Ring-membership SNARK (Trick 3)
Sybil-resistance per context	VRF output is deterministic in (sk , context)
Unlinkability across contexts	VRF output is pseudorandom across inputs
Succinct & publicly verifiable	Groth16 + Bandersnatch: constant size, fast verify



The most efficient and widely
used pairing-based SNARK
construction

The two paths to signing the same ring

First protocol (what we just described)

- Two separate proofs glued by a Pedersen commitment
- Every new signature: reprove both parts

The two paths to signing the same ring

First protocol (what we just described)

- Two separate proofs glued by a Pedersen commitment
- Every new signature: reprove both parts

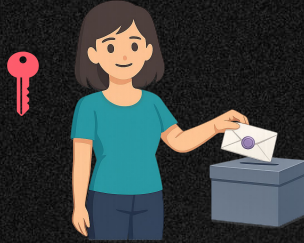
Second protocol (skipped today)

- Re-use the ring-membership proof for the same ring
- Only redo the cheap VRF evaluation proof
- Novelty of the paper: *zero-knowledge continuations*

Section 6

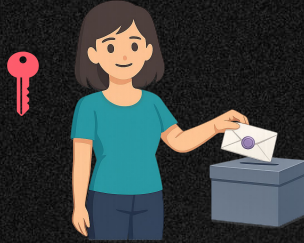
Alice revisited, and closing

Alice's vote, with all the machinery visible



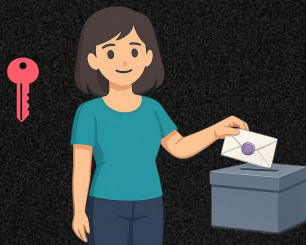
- **context** = "referendum-42" hashed onto the curve: $H(\text{context})$
- VRF output computed: $\text{Out} = \text{sk} \cdot H(\text{context})$ 

Alice's vote, with all the machinery visible



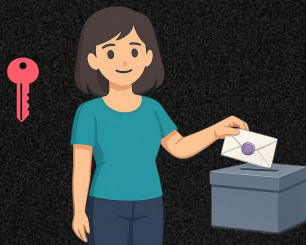
- **context** = "referendum-42" hashed onto the curve: $H(\text{context})$
- VRF output computed: $\text{Out} = \text{sk} \cdot H(\text{context})$ 
- Pedersen commitment to **sk**: $C_{\text{sk}} = \text{sk} \cdot G + r \cdot H$

Alice's vote, with all the machinery visible



- **context** = "referendum-42" hashed onto the curve: $H(\text{context})$
- VRF output computed: $\text{Out} = \text{sk} \cdot H(\text{context})$ 
- Pedersen commitment to **sk**: $C_{\text{sk}} = \text{sk} \cdot G + r \cdot H$
- VRF evaluation DLEQ proof π_{VRF} generated
- Ring-membership SNARK proof π_{ring} generated against C_{ring} 

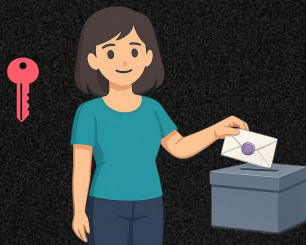
Alice's vote, with all the machinery visible



- **context** = "referendum-42" hashed onto the curve: $H(\text{context})$
- VRF output computed: $\text{Out} = \text{sk} \cdot H(\text{context})$ 
- Pedersen commitment to **sk**: $C_{\text{sk}} = \text{sk} \cdot G + r \cdot H$
- VRF evaluation DLEQ proof π_{VRF} generated
- Ring-membership SNARK proof π_{ring} generated against C_{ring}
- All of it packaged as $(\text{Out}, C_{\text{sk}}, \pi_{\text{VRF}}, \pi_{\text{ring}}) \rightarrow$ sent on-chain



Alice's vote, with all the machinery visible



- **context** = "referendum-42" hashed onto the curve: $H(\text{context})$
- VRF output computed: $\text{Out} = \text{sk} \cdot H(\text{context})$ 
- Pedersen commitment to **sk**: $C_{\text{sk}} = \text{sk} \cdot G + r \cdot H$
- VRF evaluation DLEQ proof π_{VRF} generated
- Ring-membership SNARK proof π_{ring} generated against C_{ring}
- All of it packaged as $(\text{Out}, C_{\text{sk}}, \pi_{\text{VRF}}, \pi_{\text{ring}}) \rightarrow$ sent on-chain
- Validators check; tally records Out as having voted



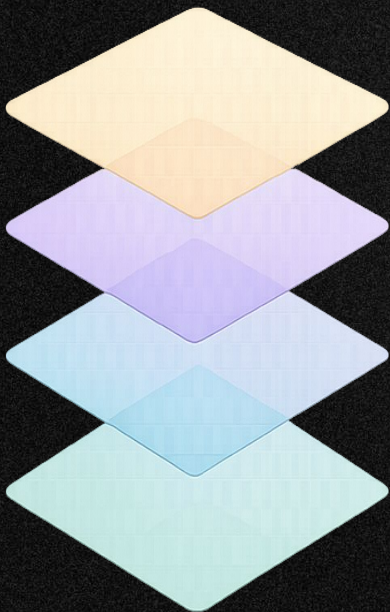
Alice's comment, for contrast

- $\text{context} = \text{"referendum-42"}$
- $\text{Out} = \text{sk} \cdot \text{H}(\text{context})$ 
- $\text{context}' = \text{"opensquare-post-17"}$
- $\text{Out}' = \text{sk} \cdot \text{H}(\text{context}')$ 

Alice's comment, for contrast

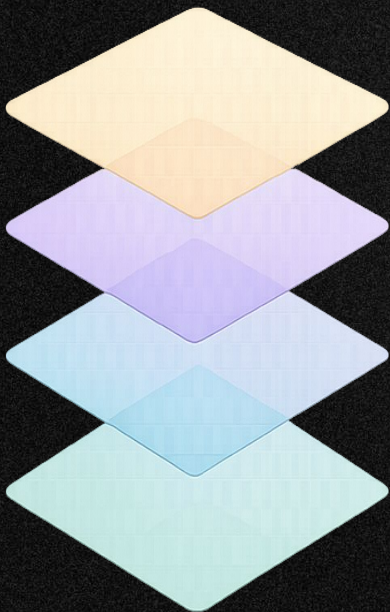
- `context = "referendum-42"`
 - `Out = sk · H(context)` 
 - `context' = "opensquare-post-17"`
 - `Out' = sk · H(context')` 
- Same ring
 - Same `sk`
 - Different context → different Out → unlinkable alias

Where this lives in Polkadot



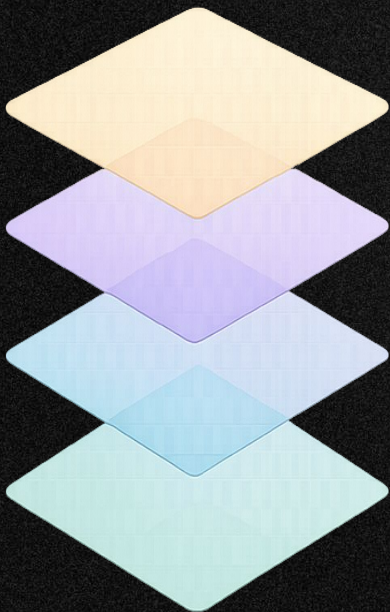
1. Crypto primitives:
 - Bandersnatch curve, KZG commitments, Groth16 SNARK

Where this lives in Polkadot



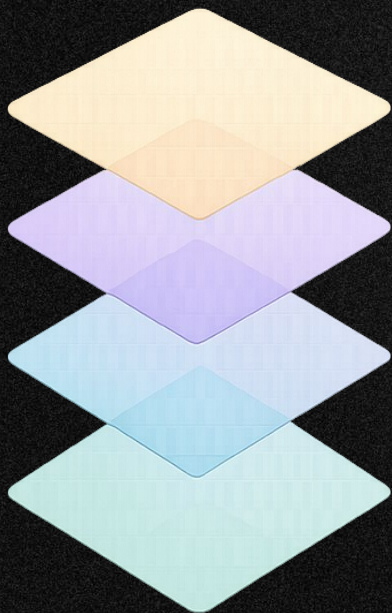
1. Crypto primitives:
 - Bandersnatch curve, KZG commitments, Groth16 SNARK
2. Ring infrastructure (`indiv-pallet-members`)
 - ring roots, onboarding, updates

Where this lives in Polkadot



1. Crypto primitives:
 - Bandersnatch curve, KZG commitments, Groth16 SNARK
2. Ring infrastructure (`indiv-pallet-members`)
 - ring roots, onboarding, updates
3. People registry (`indiv-pallet-people`, `indiv-pallet-people-lite`):
 - `PersonalId`, key lifecycle

Where this lives in Polkadot



1. Crypto primitives:
 - Bandersnatch curve, KZG commitments, Groth16 SNARK
2. Ring infrastructure (`indiv-pallet-members`)
 - ring roots, onboarding, updates
3. People registry (`indiv-pallet-people`, `indiv-pallet-people-lite`):
 - `PersonalId`, key lifecycle
4. Applications:
 - governance, treasury, resources, free usage

What we skipped

- Why DLog is hard on elliptic curves: Hardness assumption, no proof
- How KZG openings are verified: Pairings, which we didn't touch
- How Groth16 works internally: QAPs + pairings
- The trusted setup Groth16 and KZG need:
 - A one-time ceremony, Polkadot reuses the Zcash one
 - Important engineering detail, not essential to the concept
- ZK continuations: Optimisation; The paper's headline novelty

Questions?

